

pathpyG: Deep Learning using Causality-Preserving Representations of Temporal Graphs

Jürgen Hackl

HACKL@PRINCETON.EDU

*Complex Infrastructure Systems
Department of Civil and Environmental Engineering
Princeton University
Princeton, NJ, USA*

Moritz Lampert

MORITZ.LAMPERT@UNI-WUERZBURG.DE

Ingo Scholtes

INGO.SCHOLTES@UNI-WUERZBURG.DE

*Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
University of Würzburg
Würzburg, Germany*

Abstract

pathpyG is an open-source Python package for GPU-accelerated analysis of and deep learning on temporal and higher-order graphs. Building on PyTorch and PyTorch Geometric, it represents time-stamped relational data through temporal event graphs and k -th order De Bruijn graphs that encode the causality constraints induced by the timing of interactions. This higher-order representation is the basis for causality-aware graph neural network architectures for node-, edge-, and graph-level tasks. Beyond efficient GPU-accelerated model construction, **pathpyG** provides algorithms for time-respecting paths and temporal centralities, random graph models, interactive jupyter-based visualizations, tikz-based plots of graphs, as well as direct access to more than 280 data sets with more than 160,000 graphs from the **netzschleuder** repository. By bridging temporal network science and geometric deep learning in a single GPU-enabled library, **pathpyG** lowers the barrier to reproducible research and teaching.

Keywords: deep graph learning, temporal graphs, higher-order networks, De Bruijn graphs, graph neural networks

1 Introduction

Graph neural networks (GNNs) have become central to deep learning on non-Euclidean relational structures (Scarselli et al., 2009; Wu et al., 2021; Hamilton, 2020). However, many real-world systems, from social media to transportation and biology, generate time-stamped interaction data that standard graph models cannot adequately represent. Such data encode *causality constraints* – i.e. which nodes can causally influence each other given the timing of interactions – that static, time-aggregated graphs discard, leading to flawed analyses and poor learning performance (Lambiotte et al., 2019).

This issue motivates research on *higher-order causality-preserving representations of temporal graphs*. Recent work have used *higher-order De Bruijn graphs* (fig. 1) or *temporal event graphs*, whose nodes encode paths and edges capture valid consecutive transitions that form time-respecting paths (De Bruijn, 1946; Kivelä et al., 2018; Scholtes et al., 2014).

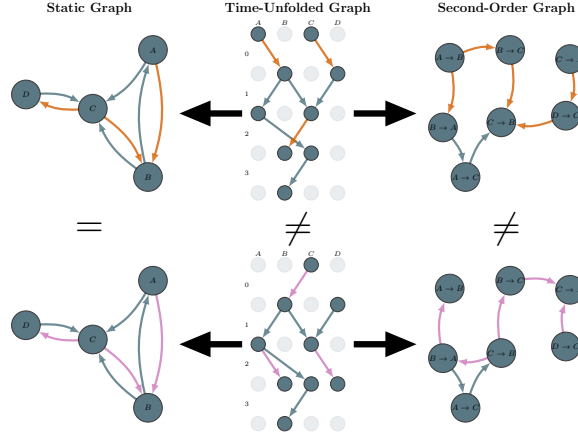


Figure 1: Static, time-unfolded, and higher-order representations from **pathpyG**. Two temporal graphs (center, top vs. bottom) yield the same static graph (left), motivating causality-preserving higher-order representations—here second-order De Bruijn graphs (right)—that distinguish them.

This enables causality-aware GNNs that improve temporal graph learning tasks (Qarkaxhija et al., 2022; Heeg and Scholtes, 2024; Heeg et al., 2025).

Yet the practical adoption of these methods is limited by a lack of accessible software that integrates causality-aware representations with deep learning. We address this gap with **pathpyG**, an open-source Python package for GPU-accelerated analysis of and deep learning on temporal graphs. Built on PyTorch and PyTorch Geometric (Paszke et al., 2019; Fey and Lenssen, 2019), it constructs temporal event graphs and k -th order De Bruijn graphs from time-stamped data, lets practitioners apply causality-aware GNNs to temporal graph learning tasks, and—through interactive Jupyter visualizations and seamless **netzschleuder** integration—lowers the barrier for researchers, educators, and practitioners in machine learning and network science.

2 Related Software Packages

pathpyG bridges two domains that existing tools address separately. In network science, temporal graph tools such as **Raphtory** (Steer et al., 2024), **TGLib** (Oettershagen and Mutzel, 2022), **Reticula** (Badie-Modiri and Kivelä, 2023), and the temporal extensions of **NetworkX** (Hagberg et al., 2008) focus on analytics without deep-learning or GPU integration, and sequential pattern miners like **SPMF** (Fournier-Viger et al., 2014) are disconnected from higher-order representations. In deep-learning, **PyTorch Geometric** (PyG) is the de facto standard (Fey and Lenssen, 2019; Fey et al., 2025) and, like **DyGLib** (Yu et al., 2023) and **TGL** (Zhou et al., 2022, 2023), models continuous-time graphs through edge sequences (Rossi et al., 2020), while **PyTorch Geometric Temporal** (Rozemberczki et al., 2021; Ockerman et al., 2025) targets regularly sampled snapshots rather than irregular data.

Unlike these edge-sequence approaches that treat time as an attribute rather than a constraint, **pathpyG**’s higher-order De Bruijn and temporal event graph representations

encode causality directly into the graph structure, letting existing PyG methods automatically respect temporal validity while leveraging GPU-accelerated model construction.

3 Package Design and Implementation

The design of `pathpyG` follows a modular structure, where the key functionality is organized in the following submodules. A typical workflow—loading a temporal graph, lifting it to a k -th order De Bruijn model that captures time-respecting paths, and training a causality-aware GNN on a GPU—requires only a few lines of code:

```
import pathpyG as pp
# load a temporal graph from the netzscheuler repository
g = pp.io.read_netschleuder_temporal_graph('sp_hospital_ward').to('cuda')
# lift to a 3rd-order De Bruijn model of time-respecting paths
# (delta = max time between consecutive events)
m = pp.MultiOrderModel.from_temporal_graph(g, delta=3600, max_order=3)
# train a causality-aware De Bruijn Graph Neural Network
model = pp.nn.DBGNN(m, num_classes=5)
```

pathpyG.core This module provides classes to represent time series data on (temporal) graphs. Its classes `Graph` and `TemporalGraph` represent static and time-stamped (directed, undirected, weighted, attributed) graphs, storing all data as `torch.tensors` following `pytorch-geometric` semantics while adding intuitive traversal (node/edge iterators, successors and predecessors) and algebraic operations (degrees, sparse adjacency and transition matrices, graph Laplacian) for spectral and random-walk methods. Graphs are built from integer edge indices or string node identifiers, transparently mapped via the `IndexMap` class. The `PathData` class stores large collections of walks, paths, or directed acyclic graphs in a single tensor, and `MultiOrderModel` adopts the multi-order model selection framework of Scholtes (2017) to construct k -th order De Bruijn graph models.

pathpyG.io This module handles the import and export of temporal graph data. It supports direct online import from the `netzscheuler` repository (more than 280 network data sets) and import/export of (attributed) temporal graphs via `pandas` data frames, and thereby formats such as CSV, JSON, SQL, or Google BigQuery.

pathpyG.visualizations This module visualizes static, temporal, and higher-order graphs through a flexible interface dispatching to several backends: `tikz` for publication-grade vector graphics, `matplotlib` for large pixel-based images, `d3.js` for interactive HTML widgets in `jupyter` notebooks, and `manim` for video visualizations of temporal data.

pathpyG.algorithms This module covers shortest (time-respecting) paths, diameter and connectivity, (temporal) graph isomorphism, and the Weisfeiler-Leman test for studying (temporal) GNN expressivity. Its central primitive is a GPU-accelerated *higher-order lifting* that extracts temporal event graphs from time-stamped data and constructs the k -th order De Bruijn graphs underlying `pathpyG`’s higher-order models. It also provides centrality measures like (temporal) betweenness and closeness or PageRank and—bridging network science and deep learning—random graph models (Erdős-Rényi, Watts-Strogatz, stochastic block, Molloy-Reed, k -regular) with randomization techniques for null-model comparisons.

data set	n	m	M	$ T $	k	event graph (s)			De Bruijn model (s)		
						CPU	GPU	$\times$	CPU	GPU	$\times$
<code>copenhagen_sms</code>	568	1,303	24,333	24,039	5	5	12	0.4	30	2	15.0
<code>sociopatterns_infectious</code>	10,972	89,034	831,824	13,526	3	9	8	1.1	40	180	0.2

Table 1: Scalability of higher-order graph construction (nodes n , unique edges m , time-stamped edges M , unique timestamps $|T|$, De Bruijn order k): CPU vs. GPU wall-clock times in seconds and speed-up ($\times$) for building event graphs and De Bruijn graphs (AMD Ryzen 9 16-core CPU, Nvidia RTX 4090 GPU).

pathpyG.statistics This module computes summary statistics—degree distributions, moments and assortativity, generating functions for degree distributions, clustering coefficients, and node similarity measures.

pathpyG.nn This module implements causality-aware message passing for temporal graphs, built on the event graph or multi-order representations. We provide the De Bruijn Graph Neural Network (DBGNN) (Qarkaxhija et al., 2022) and the Augmented Event Graph Neural Network (AuEGNN) (Heeg et al., 2025), both respecting the arrow of time and validated against their original publications; the underlying time-respecting paths run on CPU or GPU via a simple backend switch.

Installation, Documentation and Tutorials With few dependencies (`pandas`, `numpy`, `pytorch`, `pytorch-geometric`), `pathpyG` installs via `pip` from <https://github.com/pathpy/pathpyG> and ships a preconfigured *Docker* container for compute clusters. Extensive documentation and interactive *jupyter* tutorials are at <https://www.pathpy.net>.

Software Quality `pathpyG` continues the lineage of the established `pathpy` and `pathpy2` libraries (Hackl et al., 2021), which together draw close to 1,000 PyPI downloads per month. Released under the AGPL-3.0 license, it maintains code quality through a `pytest` suite run in continuous integration, `mypy` type checking, `ruff` linting, and reviewed pull requests, with versioned releases and API documentation built automatically via GitHub Actions.

Scalability Higher-order lifting exploits massively parallel tensor operations, but the benefit is data-dependent (table 1): for `copenhagen_sms`, whose many timestamps yield a large higher-order model, GPU construction of the order-5 De Bruijn model is about $15\times$ faster than the CPU, whereas for the denser `sociopatterns_infectious` data transfer to the GPU dominates. Hence GPU acceleration pays off primarily for large higher-order models.

4 Conclusions and Future Plans

By encoding the causal topology of temporal graphs, i.e. which nodes can possibly causally influence each other via time-respecting paths, into higher-order graph models, `pathpyG` serves both classical temporal network analysis and temporal graph learning within one easily accessible framework. Future work will focus on adding further causality-aware message passing architectures, integration with other graph learning libraries, and further improving scalability for large data sets.

Acknowledgments and Disclosure of Funding

Ingo Scholtes and Jürgen Hackl acknowledge support from the Swiss National Science Foundation (SNF) via grant no. 176938. Ingo Scholtes and Moritz Lampert acknowledge support from the BMBF via project TissueNet.

References

- Arash Badie-Modiri and Mikko Kivelä. Reticula: A temporal network and hypergraph analysis software package. *SoftwareX*, 21:101301, 2023. doi: 10.1016/j.softx.2022.101301. Software available at <https://github.com/reticula-network/reticula>.
- Nicolaas Govert De Bruijn. A combinatorial problem. *Proceedings of the Section of Sciences of the Koninklijke Nederlandse Akademie van Wetenschappen te Amsterdam*, 49(7):758–764, 1946.
- Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428*, 2019. Software available at <https://pyg.org/>.
- Matthias Fey, Jinu Sunil, Akihiro Nitta, Rishi Puri, Manan Shah, Blaž Stojanovič, Ramona Bendias, Alexandria Barghi, Vid Kocijan, Zecheng Zhang, et al. Pyg 2.0: Scalable learning on real world graphs. *arXiv preprint arXiv:2507.16991*, 2025.
- Philippe Fournier-Viger, Antonio Gomariz, Ted Gueniche, Azadeh Soltani, Cheng-Wei Wu, and Vincent S. Tseng. SPMF: a Java open-source pattern mining library. *Journal of Machine Learning Research*, 15(1):3569–3573, 2014. Software available at <https://www.philippe-fournier-viger.com/spmf/>.
- Jürgen Hackl, Ingo Scholtes, Luka V. Petrović, Vincenzo Perri, Luca Verginer, and Christoph Gote. Analysis and visualisation of time series data on networks with pathpy. In *Companion Proceedings of the Web Conference 2021*, WWW ’21, pages 530–532, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 978-1-4503-8313-4. doi: 10.1145/3442442.3452052.
- Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15, Pasadena, CA, USA, 2008. Software available at <https://networkx.org/>.
- William Hamilton. *Graph Representation Learning*, volume 1 of *Synthesis Lectures on Artificial Intelligence and Machine Learning*. Springer, 2020.
- Franziska Heeg and Ingo Scholtes. Using time-aware graph neural networks to predict temporal centralities in dynamic graphs, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/3514dbacaebf0f38b25adfe59ed81a8a-Paper-Conference.pdf.

- Franziska Heeg, Jonas Sauer, Petra Mutzel, and Ingo Scholtes. Weisfeiler and leman follow the arrow of time: Expressive power of message passing in temporal event graphs, 2025. URL <https://arxiv.org/abs/2505.24438>.
- Mikko Kivelä, Jordan Cambe, Jari Saramäki, and Márton Karsai. Mapping temporal-network percolation to weighted, static event graphs. *Scientific reports*, 8(1):12357, 2018.
- Renaud Lambiotte, Martin Rosvall, and Ingo Scholtes. From networks to optimal higher-order models of complex systems. *Nature physics*, 15(4):313–320, 2019.
- Seth Ockerman, Amal Gueroudji, Tanwi Mallick, Yixuan He, Line Pouchard, Robert Ross, and Shivaram Venkataraman. Pgt-i: Scaling spatiotemporal gnns with memory-efficient distributed training, 2025. URL <https://arxiv.org/abs/2507.11683>.
- Lutz Oettershagen and Petra Mutzel. TGLib: An open-source library for temporal graph analysis. In *IEEE International Conference on Data Mining Workshops, ICDM 2022*, pages 1240–1245. IEEE, 2022. doi: 10.1109/ICDMW58026.2022.00160. URL <https://doi.org/10.1109/ICDMW58026.2022.00160>. Software available at <https://gitlab.com/tgpublic/tglib>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- Lisi Qarkaxhija, Vincenzo Perri, and Ingo Scholtes. De Bruijn goes neural: Causality-aware graph neural networks for time series data on dynamic graphs. In *Learning on Graphs Conference*, pages 51–1. PMLR, 2022.
- Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. Temporal graph networks for deep learning on dynamic graphs. *CoRR*, abs/2006.10637, 2020. URL <https://arxiv.org/abs/2006.10637>.
- Benedek Rozemberczki, Paul Scherer, Yixuan He, George Panagopoulos, Alexander Riedel, Maria Astefanoaei, Oliver Kiss, Ferenc Beres, , Guzman Lopez, Nicolas Collignon, and Rik Sarkar. PyTorch Geometric Temporal: Spatiotemporal Signal Processing with Neural Machine Learning Models. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*, page 4564–4573, 2021. Software available at <https://pytorch-geometric-temporal.readthedocs.io/>.
- Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The Graph Neural Network Model. *IEEE Transactions on Neural Networks*, 20(1):61–80, January 2009. ISSN 1941-0093. doi: 10.1109/TNN.2008.2005605.
- Ingo Scholtes. When is a Network a Network? Multi-Order Graphical Model Selection in Pathways and Temporal Networks. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’17*, pages 1037–1046,

New York, NY, USA, August 2017. Association for Computing Machinery. ISBN 978-1-4503-4887-4. doi: 10.1145/3097983.3098145.

Ingo Scholtes, Nicolas Wider, Rene Pfitzner, Antonios Garas, Claudio Juan Tessone, and Frank Schweitzer. Causality-driven slow-down and speed-up of diffusion in non-markovian temporal networks. *Nature Communications*, 5:5024, September 2014. doi: 10.1038/ncomms6024. URL <http://www.nature.com/ncomms/2014/140924/ncomms6024/full/ncomms6024.html>. <https://doi.org/10.1038/ncomms6024>.

Ben Steer, Naomi A. Arnold, Cheick Ba Tidiane, Renaud Lambiotte, Haaron Yousaf, Lucas G. S. Jeub, Fabian Murariu, Shivam Kapoor, Pedro Rico, Rachel Chan, Louis Chan, James Alford, and Richard G. Clegg. Raphtory: The temporal graph engine for Rust and Python. *Journal of Open Source Software*, 9(95):5940, 2024. doi: 10.21105/joss.05940. Software available at <https://github.com/Pometry/Raphtory>.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, January 2021. ISSN 2162-2388. doi: 10.1109/TNNLS.2020.2978386.

Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. Towards better dynamic graph learning: New architecture and unified library. In *NeurIPS*, 2023. Software available at <https://github.com/yule-BUAA/DyGLib>.

Hongkuan Zhou, Da Zheng, Israt Nisa, Vasileios Ioannidis, Xiang Song, and George Karypis. TGL: A general framework for temporal gnn training on billion-scale graphs. *Proc. VLDB Endow.*, 15(8), 2022. Software available at <https://github.com/amazon-science/tgl>.

Hongkuan Zhou, Da Zheng, Xiang Song, George Karypis, and Viktor K. Prasanna. Disttgl: Distributed memory-based temporal graph neural network training. In *SC*, pages 39:1–39:12. ACM, 2023.